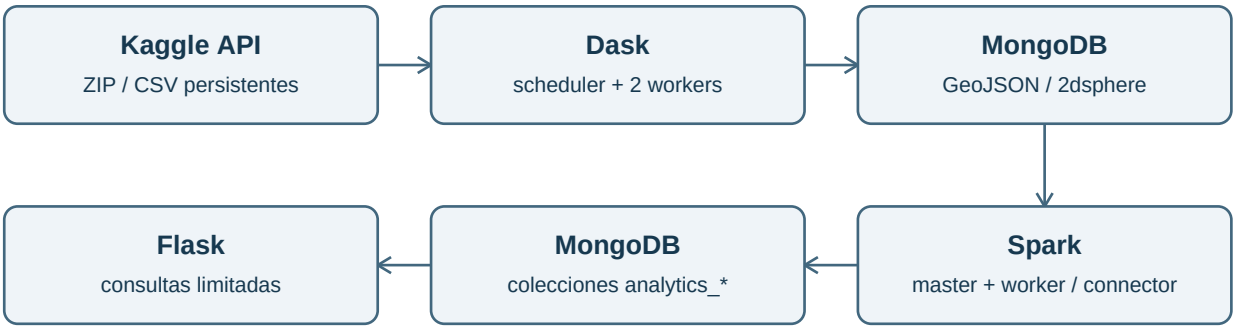
US Accidents Procesamiento geoespacial y CI/CD

Informe técnico académico · 2 de octubre de 2026
Repositorio: DavierGonz/ParcialBigData. Fuente de requisitos: guía Trabajo_Big_Data_Geoespacial-1 del profesor. Evidencias detalladas: docs/matriz-final.md e informes de fases 1-5.

Propósito y alcance

Se implementó un flujo reproducible para descargar, limpiar, almacenar, agregar y consultar accidentes georreferenciados. La muestra final contiene **1.103.657 documentos únicos**. La solución ejecuta trabajo distribuido dentro de Docker Desktop en un solo Windows; no pretende demostrar un clúster de máquinas físicas ni emplea SIG adicional o machine learning.



GitHub -> HTTPS / HMAC -> Jenkins -> pruebas -> candidata -> despliegue Flask

Compose: red interna, puertos localhost, volúmenes persistentes y healthchecks.

Arquitectura y versiones

MongoDB 8.0.16; Dask/distributed 2025.5.1; Spark 3.5.8 con MongoDB Spark Connector 10.7.0, Scala 2.12 y Java 17; Flask 3.1.3; Jenkins 2.568.3 JDK21. Imágenes base fijadas por digest, dependencias Python fijadas y entorno local .venv separado de los contenedores.

El host dispone de 16 GB RAM y Docker de aproximadamente 7,64 GiB. Los ocho servicios base tienen límites que suman 6,875 GiB. La pasarela webhook y el túnel añaden 256 MiB de presupuesto. Las cargas pesadas Dask y Spark se ejecutan secuencialmente; el benchmark sustituye temporalmente los servicios de cómputo.

Datos, limpieza e ingesta distribuida

Fuente: US Accidents, Sobhan Moosavi, Kaggle, versión 13 (actualización registrada 2023-05-28), licencia CC BY-NC-SA 4.0. La descarga completa usa API desde Jenkins con credencial protegida. Se conserva ZIP, CSV y manifiesto de versión, esquema, tamaños y SHA-256; los datos no se publican en Git.

Magnitud comprobada (Fase 3)	Valor
ZIP descargado	684.855.912 bytes
CSV extraído / columnas	3.058.183.727 bytes / 46
Filas recorridas	7.728.394
Descartes reales de coordenadas	0
No seleccionadas por muestreo	6.624.737
Muestra limpia única almacenada	1.103.657

Reglas y muestra

Se usan Start_Lat y Start_Lng. Se rechazan valores ausentes, no numéricos/no finitos y coordenadas fuera de [-90,90] / [-180,180], en categorías excluyentes. ID ausente también impide la carga. Cero y extremos son válidos; no se elimina un punto por una caja aproximada de EE. UU. Los fixtures sí incluyen entradas inválidas, aunque la fuente real no produjo descartes de coordenadas.

La selección determinista usa BLAKE2b de 64 bits sobre us-accidents-academic-v1:<ID>, módulo 7 igual a cero. Recorre todo el archivo y no toma simplemente las primeras filas. Se exige al menos un millón de documentos únicos; no se afirma representatividad estadística por estado.

Ejecución y persistencia

Dask procesa 182 particiones de 16 MiB con máximo dos en vuelo. Cada worker limpia y escribe lotes de 500 ReplaceOne/upsert; el coordinador recibe contadores, no el CSV completo. _id=ID evita duplicados. La huella incluye versión, archivo, ETL y muestreo. Un bloqueo compartido evita solapar ingesta y Spark.

Primera ejecución histórica: descarga 74,315 s, ETL 106,216 s y operación total 186,659 s; build Jenkins 261,421 s. La repetición forzada encontró los mismos 1.103.657 registros sin insertar ni modificar documentos. Caché íntegra y estado completo permiten omitir descarga/ETL posteriores. No se implementa sustitución atómica entre versiones distintas.

MongoDB y procesamiento Spark

Geometría y consultas

Cada accidente conserva columnas fuente y location={type: Point, coordinates: [longitud, latitud]}. MongoDB aplica validador GeoJSON e índice 2dsphere. $near ordena proximidad; $geoWithin filtra interior de un Polygon; una agregación empieza explícitamente con $geoNear para devolver distancia en metros. Entradas parametrizadas, límites y tiempo máximo evitan consultas masivas.

Agregaciones distribuidas

Spark lee ocho particiones reales mediante el conector MongoDB, filtrando la huella de muestra y proyectando campos necesarios. Persiste intermedios DISK_ONLY. El driver recibe contadores y comprobaciones pequeñas; no colecta el dataset completo. Grilla angular: x=min(3599,floor((lon+180)*10)); y=min(1799,floor((lat+90)*10)).

Colección nueva	Resultado comprobado
analytics_grid	23.599 celdas; suma 1.103.657
analytics_time	24 horas, 2.549 fechas, 87 meses: 2.660 grupos
analytics_hotspots	50 celdas con mayor frecuencia
analytics_runs / analytics_state	1 ejecución completa / 1 puntero publicado

Cada granularidad temporal suma 1.103.657; fechas no interpretables: cero en la muestra. Los tiempos son civiles del archivo, sin inventar UTC. La principal celda, (617,1240), contiene 7.176 accidentes: longitud -118,3 a -118,2 y latitud 34,0 a 34,1. Es concentración de observaciones, no tasa de riesgo; celdas angulares no tienen área constante.

Idempotencia y trazabilidad

Claves deterministas de ejecución/grupo y upserts de 250 evitan duplicados. Se verifican totales antes de publicar el puntero activo; la API no expone corridas incompletas. La ejecución analítica conserva versión, huella, método y conteos. Reutilizar resultados completos requiere comprobar consistencia.

Fase 4: procesamiento completo 120,162 s (128,016 s del comando); reutilización 2,686 s. Máximo observado simultáneo de ocho servicios: 2.770,23 MiB; MongoDB alcanzó 726 MiB de su límite de 768 MiB, con presión de memoria pero sin OOM kill. Muestras aproximadamente cada cinco segundos: no son un máximo absoluto del host.

API Flask y comprobación funcional

Ruta / método	Entrada y resultado
GET /health	Salud del proceso Flask; no certifica los motores
GET /accidents/near	lat, lon, radius_m, limit; documentos por $near
GET /accidents/geo-near	Mismos parámetros; $geoNear y distance_m
POST /accidents/within	JSON geometry Polygon y limit; $geoWithin
GET /analytics	kind=grid|hotspots|hour|day|month; limit, offset

Latitud/longitud finitas y en rango; radio 1-50.000 m; limit por defecto 20 y máximo 100. Offset analítico máximo 10.000. Polygon admite un anillo cerrado de 3-100 vértices distintos, sin huecos, autointersecciones ni antimeridiano, con extensión máxima de 5° por eje. Cuerpo máximo 32 KiB; consultas MongoDB máximo 3 s. Se lee limit+1 para informar has_more.

Respuestas y errores

HTTP 200 correcto; 400 entrada inválida; 404 ruta inexistente; 405 método; 413 cuerpo excesivo; 415 contenido; 503 base no disponible o analítica no publicada; 504 timeout. Los errores no exponen credenciales. Contrato completo y ejemplos CMD en docs/api-geoespacial.md y README.

Ejemplo real: lat=32.81517, lon=-116.914566, radius_m=1000, limit=2 devuelve A-44917 y A-71824. $geoNear informa 0,0 y 447,737315 m. Hotspots con limit=1 devuelve la celda principal y count=7176. La inclusión por polígono no promete orden de proximidad.

Pruebas con datos conocidos y reales

Fixtures aislados con UUID comprueban distancias, interior/exterior, límites, publicación y paginación. Spark prueba fronteras geográficas, fechas inválidas y doble escritura. Dask prueba limpieza, particiones, carga y repetición en ambos workers. La coherencia real exige la misma huella y total en cada granularidad; el benchmark compara todas las tuplas de celdas, no solo la suma.

Validación final previa a publicar: 62 pruebas en la imagen API, 2 de Spark y 4 API/MongoDB pasaron antes del ensayo de rollback. El fallo HTTP posterior fue deliberado y Jenkins restauró la imagen anterior. El ensayo pytest separado pasó 62 y falló una prueba deliberada, sin cambiar contenedor ni imagen de producción.

Jenkins, GitHub y reproducibilidad

Pipeline final: checkout privado real → build → pytest → integración de servicios → Spark y API/MongoDB → construir candidata → HTTP → despliegue → HTTP. El job de ingesta separado automatiza Kaggle y tiene sus propias barreras. Un ejecutor serializa jobs; las credenciales se inyectan desde Jenkins. El socket Docker exige ejecutar solo código de confianza.

Promoción y recuperación

Las marcas de etapas exitosas bloquean promoción incompleta. Se captura la imagen anterior; un fallo posterior la restaura y verifica salud. Si rollback falla, un marcador bloquea nuevos builds. En la verificación final, snapshot #1 falló pytest (73,588 s) sin despliegue; #2 provocó fallo posterior (99,127 s) y acreditó rollback exacto. Snapshot #3 terminó SUCCESS (95,471 s), 68 pruebas aprobadas y promoción verificada.

Integración segura y límite de aprobación

Jenkins usa deploy key SSH de solo lectura y claves de host GitHub verificadas. GitHub alcanza una pasarela restringida por túnel HTTPS saliente, sin abrir router. Pasarela y Jenkins validan HMAC-SHA256; se filtran repositorio/ramas y deduplican entregas. La administración no se reenvía. Un ping real de GitHub respondió HTTP 200; accesos públicos a /login y /scriptText dieron 404 y webhook sin firma dio 403.

El checkout privado de main se comprobó en un build de solo lectura, separado de los ensayos completos de snapshot. El build completo del commit publicado y su evento real se registran en el PR final y artefactos Jenkins. Este documento no atribuye un merge a main que todavía requiere aprobación. Quick Tunnel cambia de URL al reiniciar: el helper actualiza el webhook; no ofrece disponibilidad permanente.

Arranque limpio y evidencias

README documenta todo el arranque. La reproducción usó otro checkout con fuentes candidatas identificadas, venv, contraseñas y volúmenes nuevos: descarga independiente 95,388 s, ETL 101,346 s y Spark 95,981 s. Obtuvo la misma muestra y agregados; HTTP pasó. Es el mismo Windows, con caché de software compartida, no un segundo equipo físico. Detalles: docs/informe-final.md.

Secretos, .env, .venv, datasets, cachés y logs locales permanecen fuera de Git. Se conservan métricas pequeñas publicables y se audita contenido antes de publicar. La simulación de sustentación propone agregar GET /demo, probar, hacer commit/PR/merge y comprobar webhook/Jenkins/HTTP; no se creó el commit de demostración.

Benchmark: resultados y límites

Operación común: grilla 0,1° sobre exactamente 1.103.657 accidentes. Ambos motores leen ocho CSV con idénticos bytes y tipos; la proyección ocupa 60.774.261 bytes. Se excluye preparación desde MongoDB (7,207 s), escritura persistente y creación del clúster. Cachés no se vacían. Cada repetición crea nueva aplicación/cliente; el total incluye arranque, lectura, agregación, validación y cierre.

C1/C2: uno/dos workers por motor, 1 CPU y contenedor de 1.536 MiB por worker; proceso Dask/heap ejecutor Spark 768 MiB. Coordinador/driver: 1 CPU y 1.024 MiB. Cuotas exteriores iguales; proceso Python y heap JVM no son medidas idénticas. Cómputo productivo detenido, presupuesto concurrente 4,5/6 GiB. Un calentamiento y tres mediciones por combinación, siempre secuenciales.

Motor / workers	Total mediana s	Operación s	Pico motor MiB	CPU s mediana
Dask / 1	3,022	1,752	424,9	3,074
Dask / 2	2,384	1,034	575,7	3,458
Spark / 1	16,344	11,409	919,1	19,962
Spark / 2	15,153	10,065	1.306,5	26,219

Las 16 ejecuciones, incluidos cuatro calentamientos, generaron exactamente 23.599 celdas y suma 1.103.657, con igualdad de todas las tuplas ordenadas. Cero errores, reintentos y OOM kills en la serie formal. Pico simultáneo observado de todos los contenedores: 2.314,19 MiB. Memoria muestreada aproximadamente cada segundo; CPU acumulada por cgroup. No incluye toda la VM/Windows.

Conclusión estrictamente experimental

En esta operación, muestra y equipo, Dask fue más rápido y usó menos memoria observada. Dos workers Dask redujeron mediana total 21,1% frente a uno, aumentando memoria observada 35,5%. Dos workers Spark redujeron mediana 7,3%, con aumento de memoria 42,1%; rangos se solapan. Tres repeticiones no demuestran significancia estadística. No hay un escenario medido en que Spark gane ni evidencia para extrapolar a datos mayores, joins, clúster físico o sesiones persistentes.

Fuentes y evidencia ampliada

Guía original: Trabajo_Big_Data_Geoespacial-1.docx (hash en matriz). Dataset: kaggle.com/datasets/sobhanmoosavi/us-accidents. Repositorio: github.com/DavierGonz/ParcialBigData (privado). Informes fases 1-5 y docs/benchmark/{results,summary,manifest,resources}.json conservan metodología, repeticiones y métricas. docs/matriz-final.md vincula requisitos, código y pruebas; README documenta reproducción; PR final y Jenkins conservan validación de publicación.